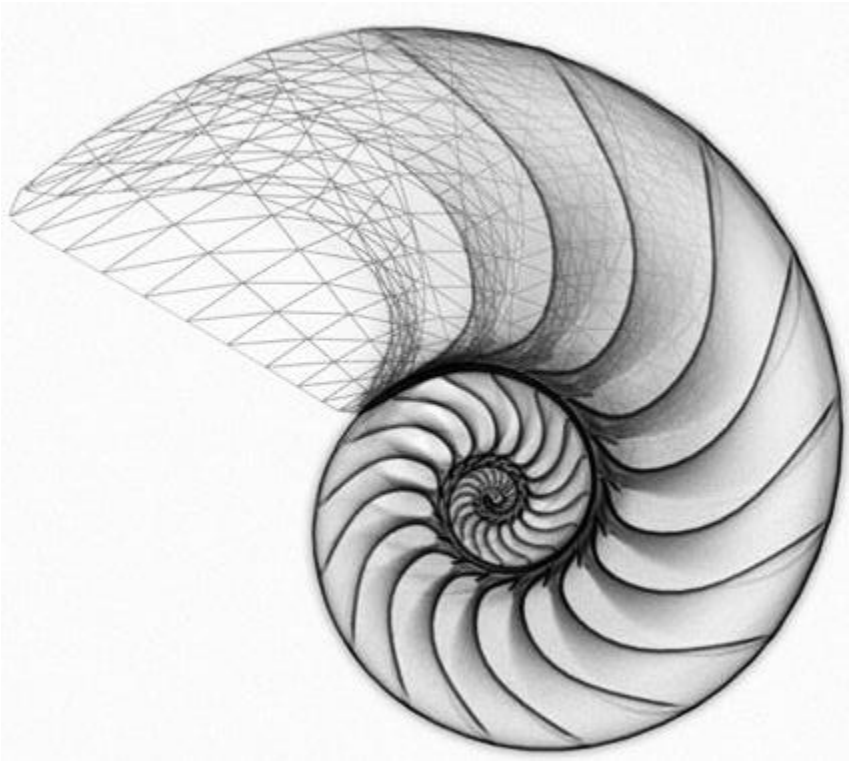


Lecture 02

Dynamic Memory Allocation

CSE225: Data Structures and Algorithms

Arrays and Pointers

```
#include <stdio.h>

int main(void)
{
     char str[5] = {'H', 'E', 'L', 'L', 'O'};
     char *ptr = &str[0];
     printf("ptr = %08x\n", ptr);
     printf("str = %08x\n", str);
     return 0;
}
```

Arrays and Pointers

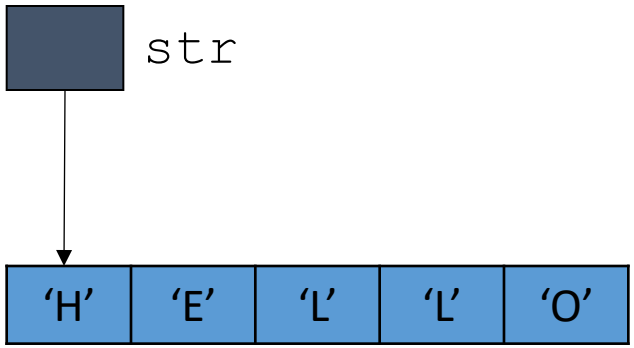
```
#include <stdio.h>

int main(void)
{
     char str[5] = {'H', 'E', 'L', 'L', 'O'};
     char *ptr = &str[0];
     printf("ptr = %08x\n", ptr);
     printf("str = %08x\n", str);
     return 0;
}
```

Output:

```
ptr = 0028ff17
str = 0028ff17
```

Arrays and Pointers

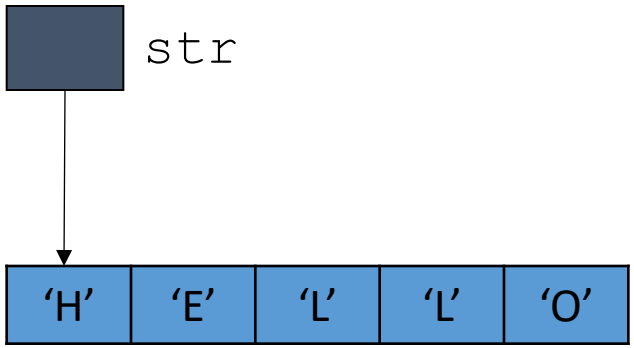
The array name is basically the name of a pointer variable which contains the starting address of the array (address of the first element)



	address	content
	0x00000000	
	0x00000001	
	.	
	.	
str	0x180A96e7	0x180A96f3
	0x180A96e8	
	0x180A96e9	
	0x180A96f0	
	0x180A96f1	
	0x180A96f2	
	0x180A96f3	'H'
	0x180A96f4	'E'
	0x180A96f5	'L'
	0x180A96f6	'L'
	0x180A96f7	'O'
	.	
	.	

Arrays and Pointers

The array name is basically the name of a pointer variable which contains the starting address of the array (address of the first element)



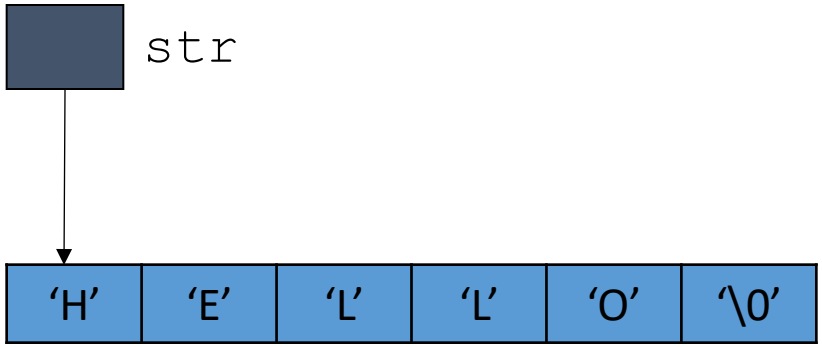
`c = str[2];` is equivalent to

`c = *(str + 1 × 2);`

Base offset

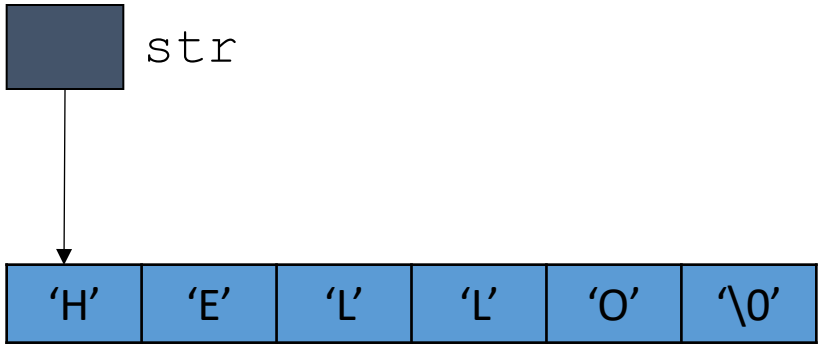
	address	content
	0x00000000	
	0x00000001	
	.	
	.	
str	0x180A96e7	0x180A96f3
	0x180A96e8	
	0x180A96e9	
	0x180A96f0	
	0x180A96f1	
	0x180A96f2	
	0x180A96f3	'H'
	0x180A96f4	'E'
	0x180A96f5	'L'
	0x180A96f6	'L'
	0x180A96f7	'O'
	.	
	.	

Arrays and Pointers



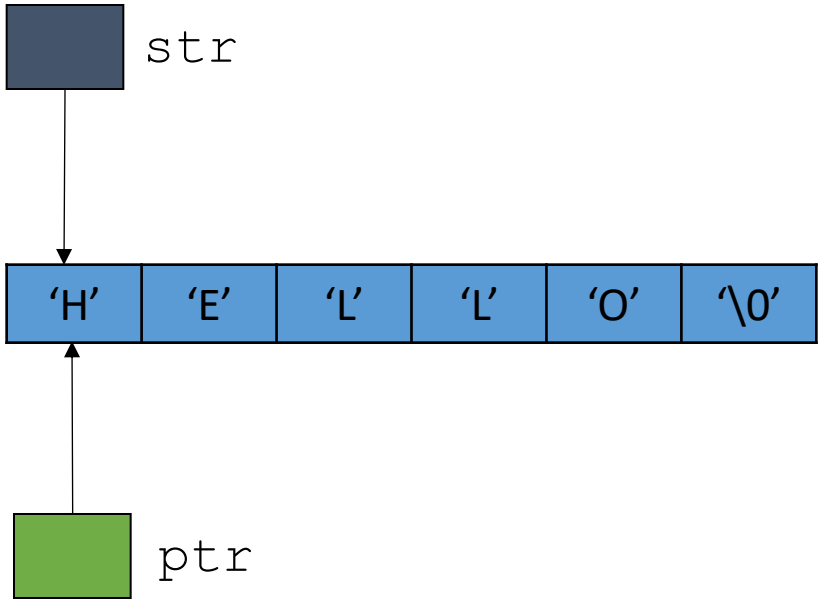

```
char str[6] = “HELLO";
```

Arrays and Pointers



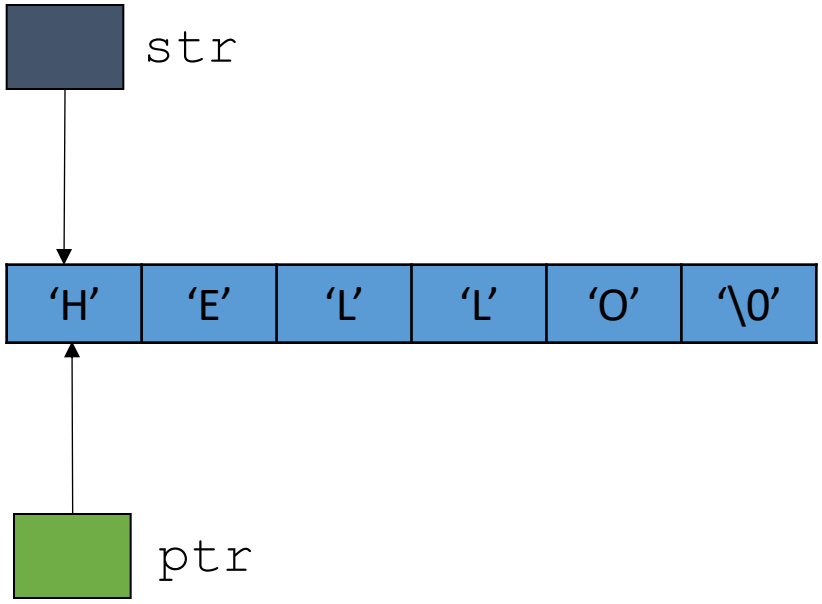

```
char str[6] = “HELLO";
char *ptr = str;
```

Arrays and Pointers



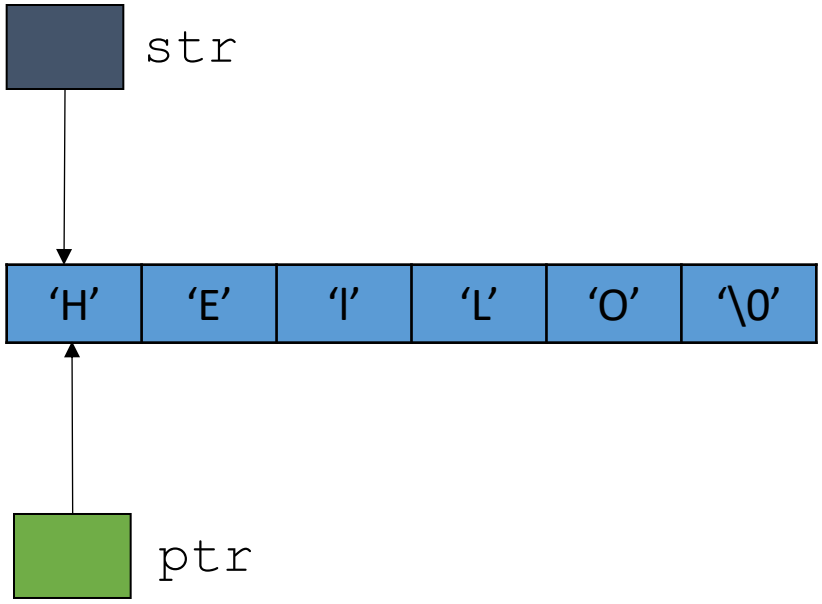

```
char str[6] = "HELLO";
char *ptr = str;
```

Arrays and Pointers



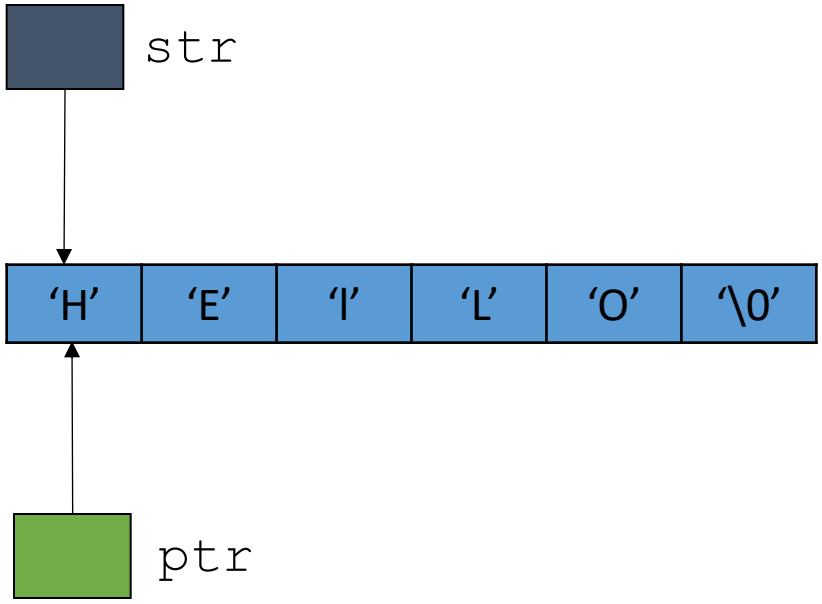

```
char str[6] = "HELLO";
char *ptr = str;
ptr[2] = 'l';
```

Arrays and Pointers



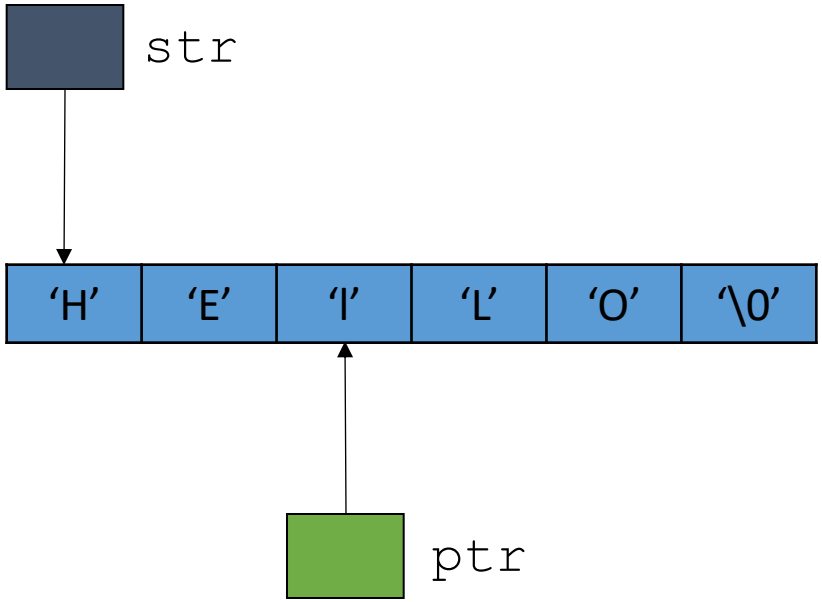

```
char str[6] = "HELLO";
char *ptr = str;
ptr[2] = 'l';
```

Arrays and Pointers




```
char str[6] = "HELLO";
char *ptr = str;
ptr[2] = 'l';
ptr = ptr + 2;
```

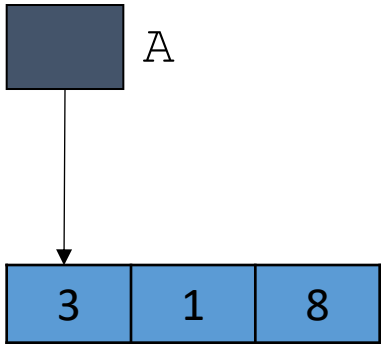
Arrays and Pointers




```
char str[6] = "HELLO";
char *ptr = str;
ptr[2] = 'l';
ptr = ptr + 2;
```

Arrays and Pointers

```
int A[3] = {3, 1, 8};
```



`i = A[2];` is equivalent to

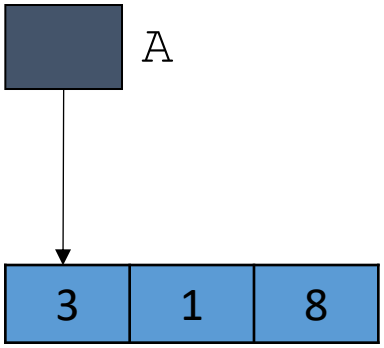
`i = *(A + 4 × 2);`

Base offset

	address	content
	0x00000000	
	0x00000001	
	.	
	.	
A	0x180A96e7	0x180A96f3
	0x180A96e8	
	0x180A96e9	
	0x180A96f0	
	0x180A96f1	
	0x180A96f2	
	0x180A96f3	3
	0x180A96f4	
	0x180A96f5	
	0x180A96f6	
	0x180A96f7	1
	0x180A96f8	
	0x180A96f9	
	0x180A96fA	
	0x180A96fB	8
	0x180A96fC	
	0x180A96fD	
	0x180A96fE	

Arrays and Pointers

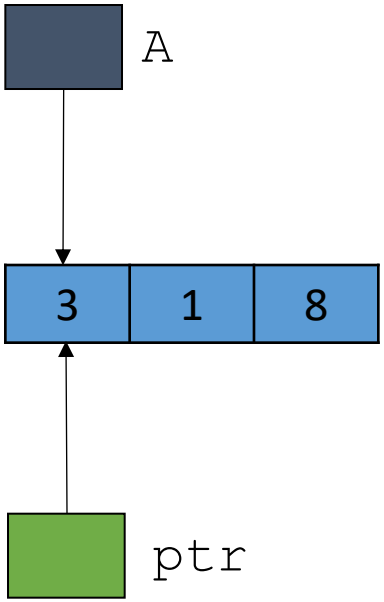
```
int A[3] = {3, 1, 8};
```




```
int *ptr = A;
```

Arrays and Pointers

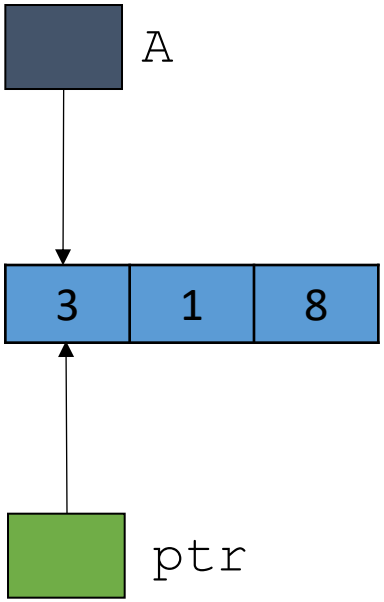
```
int A[3] = {3, 1, 8};
```




```
int *ptr = A;
```

Arrays and Pointers

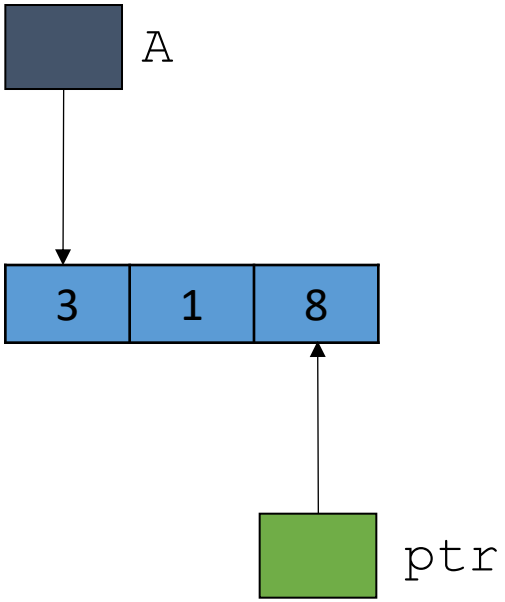
```
int A[3] = {3, 1, 8};
```




```
int *ptr = A;
ptr = ptr + 2;
```

Arrays and Pointers

```
int A[3] = {3, 1, 8};
```



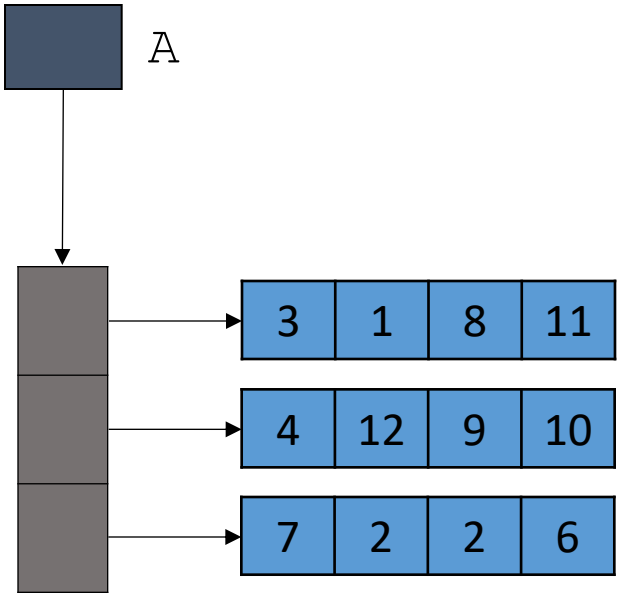

```
int *ptr = A;
ptr = ptr + 2;
```

Arrays and Pointers

```
int A[3][4] = {{3, 1, 8, 11}, {4, 12, 9, 10}, {7, 5, 2, 6}};
```

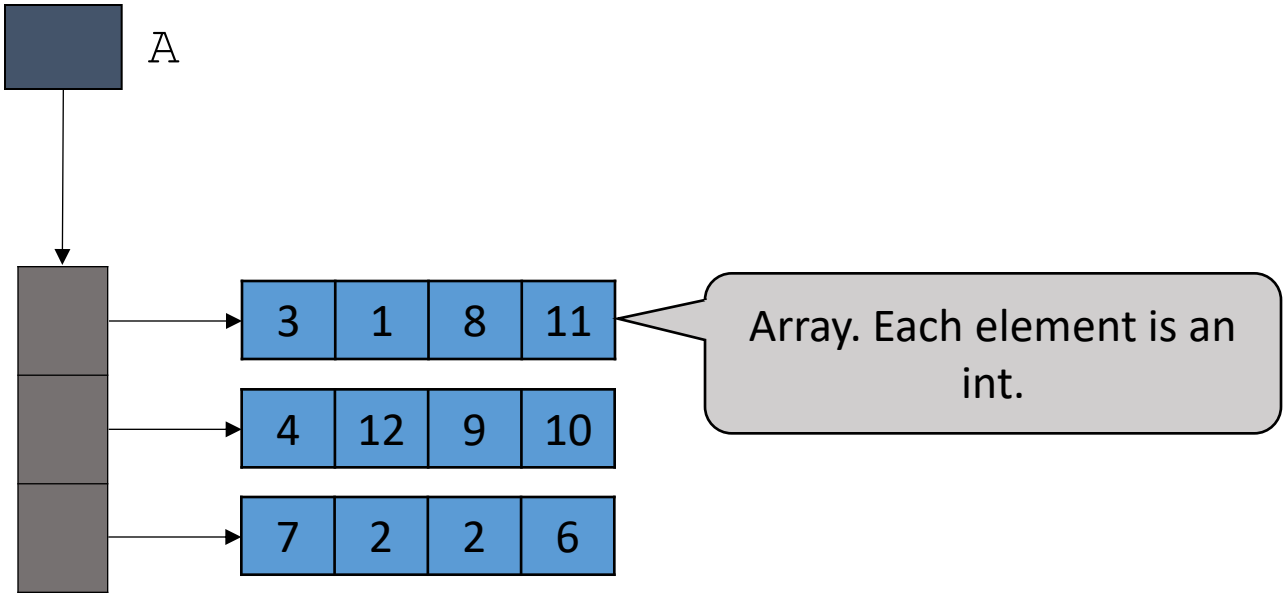
Arrays and Pointers

```
int A[3][4] = {{3, 1, 8, 11}, {4, 12, 9, 10}, {7, 5, 2, 6}};
```



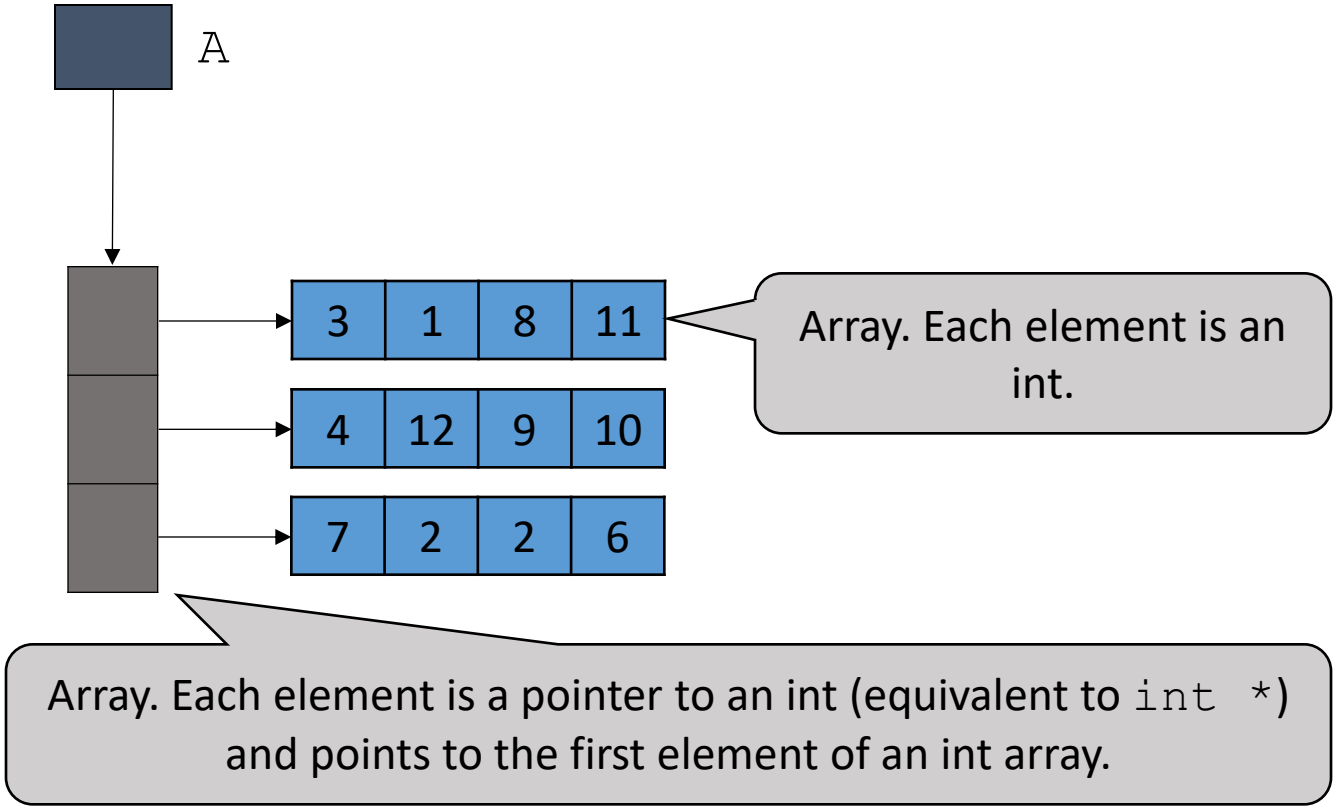
Arrays and Pointers

```
int A[3][4] = {{3, 1, 8, 11}, {4, 12, 9, 10}, {7, 5, 2, 6}};
```



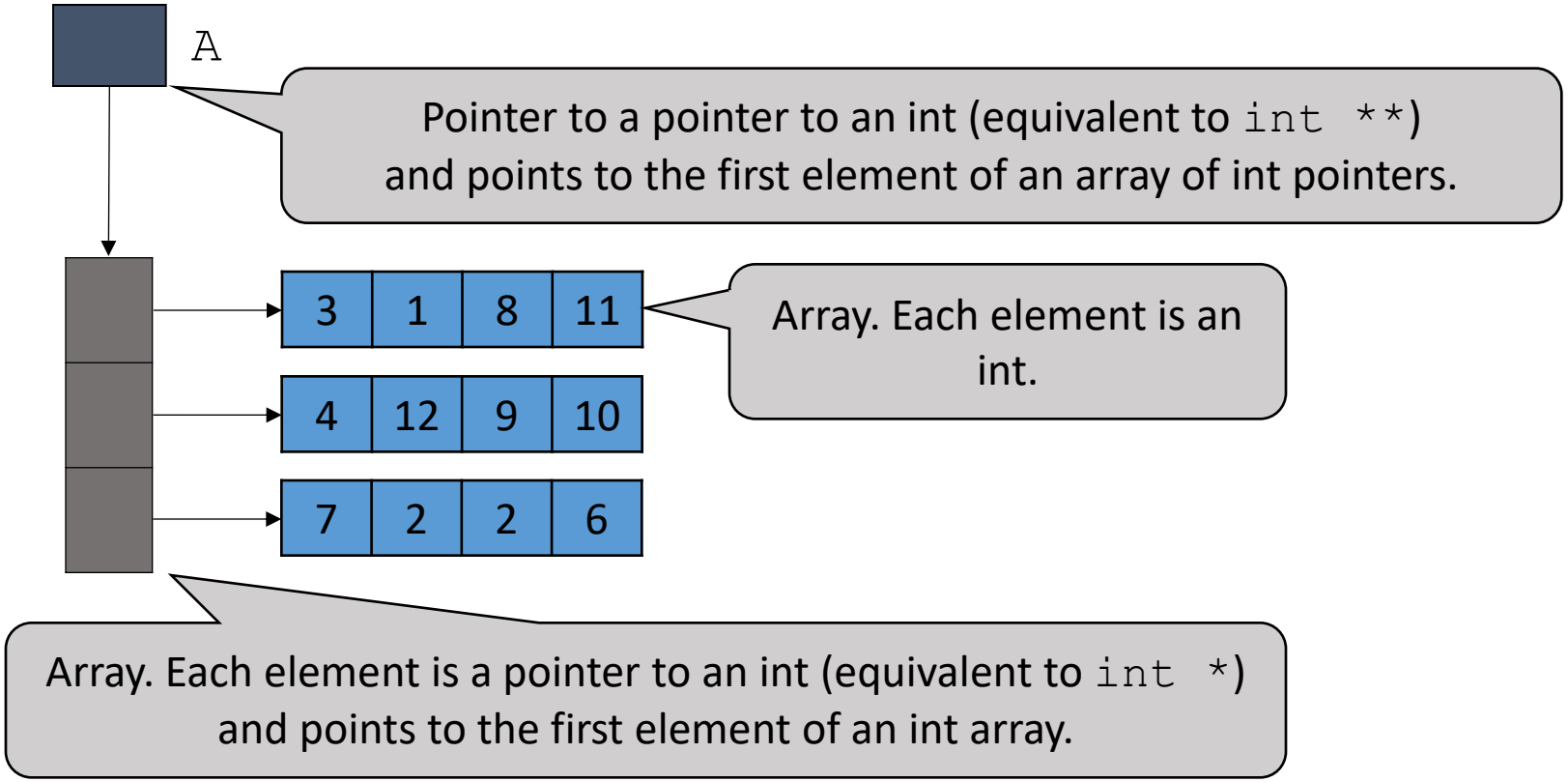
Arrays and Pointers

```
int A[3][4] = {{3, 1, 8, 11}, {4, 12, 9, 10}, {7, 5, 2, 6}};
```



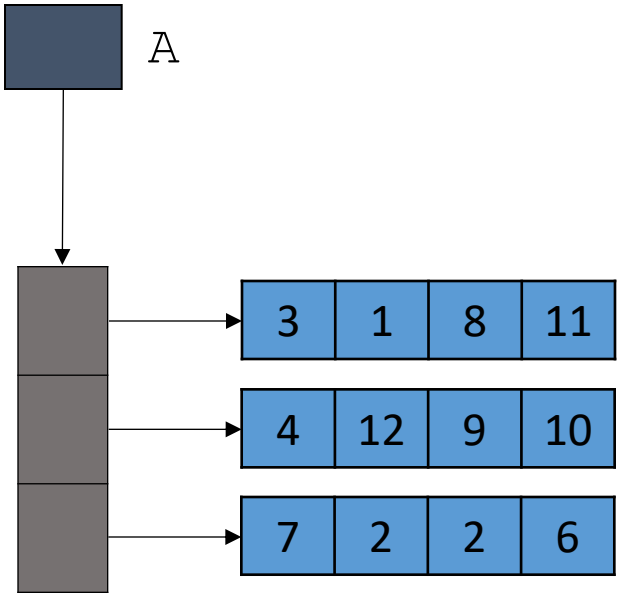
Arrays and Pointers

```
int A[3][4] = {{3, 1, 8, 11}, {4, 12, 9, 10}, {7, 5, 2, 6}};
```



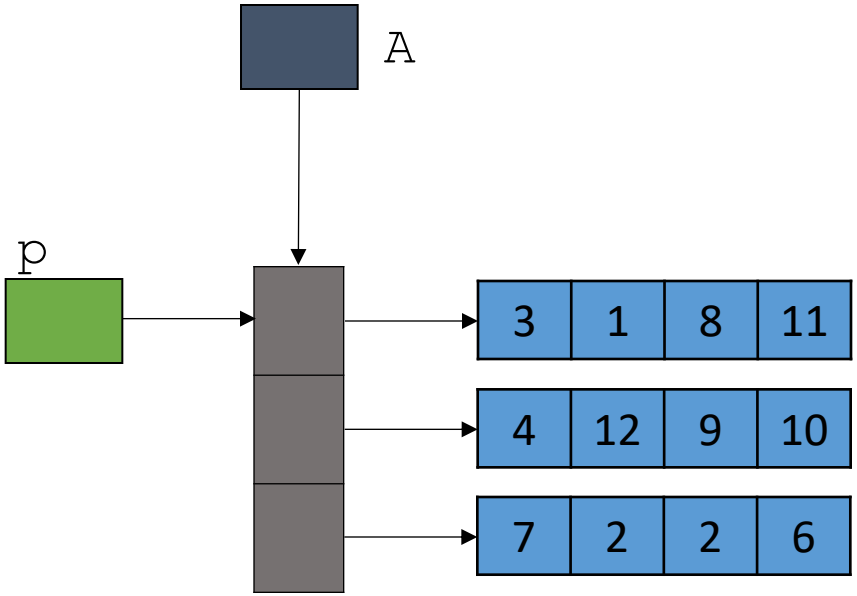
Arrays and Pointers

```
int A[3][4] = {{3, 1, 8, 11}, {4, 12, 9, 10}, {7, 5, 2, 6}};
int (*p)[4] = A;
```



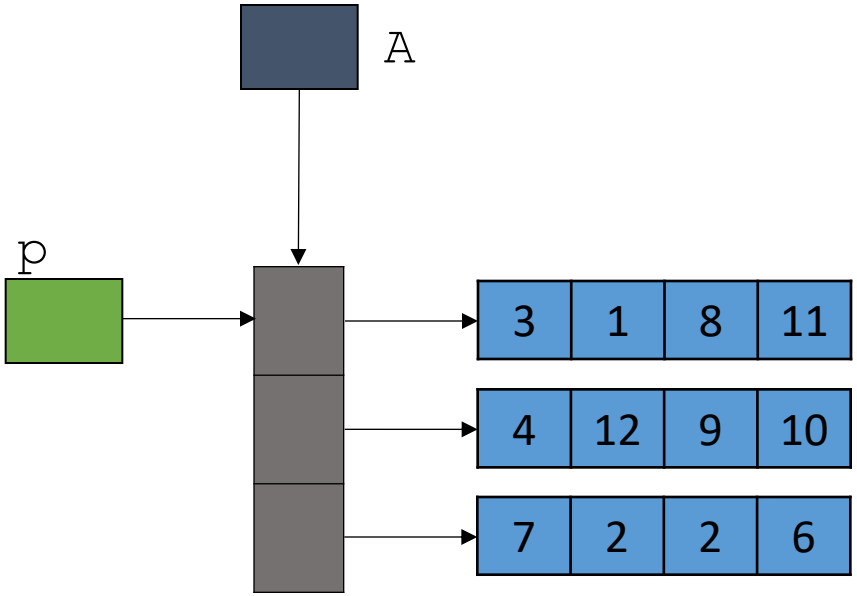
Arrays and Pointers

```
int A[3][4] = {{3, 1, 8, 11}, {4, 12, 9, 10}, {7, 5, 2, 6}};
int (*p)[4] = A;
```



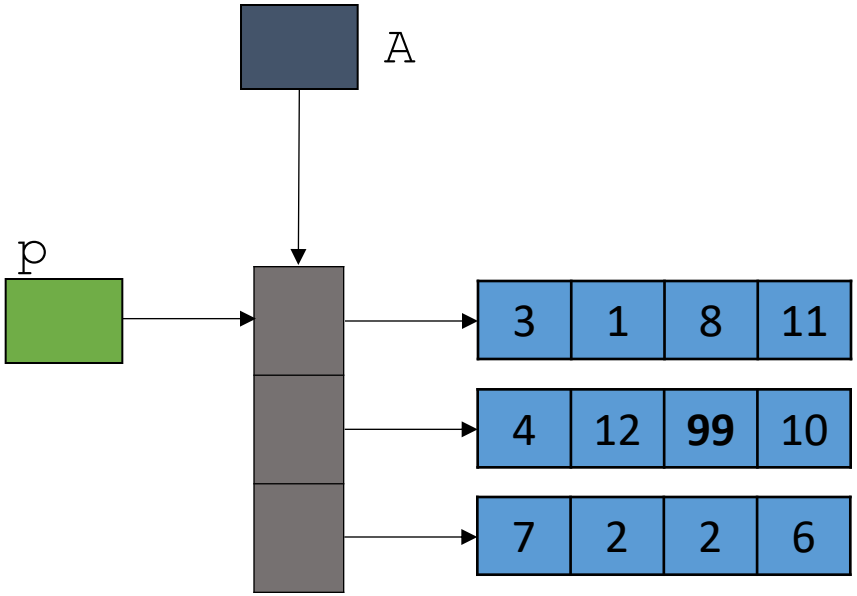
Arrays and Pointers

```
int A[3][4] = {{3, 1, 8, 11}, {4, 12, 9, 10}, {7, 5, 2, 6}};
int (*p)[4] = A;
P[1][2] = 99;
```



Arrays and Pointers

```
int A[3][4] = {{3, 1, 8, 11}, {4, 12, 9, 10}, {7, 5, 2, 6}};
int (*p)[4] = A;
P[1][2] = 99;
```



Allocation of Memory

- ***Static Allocation*:** Amount of memory space required is determined in advance (that is, at the compilation time) and memory space is allocated to the program right before the program is executed.

For example,

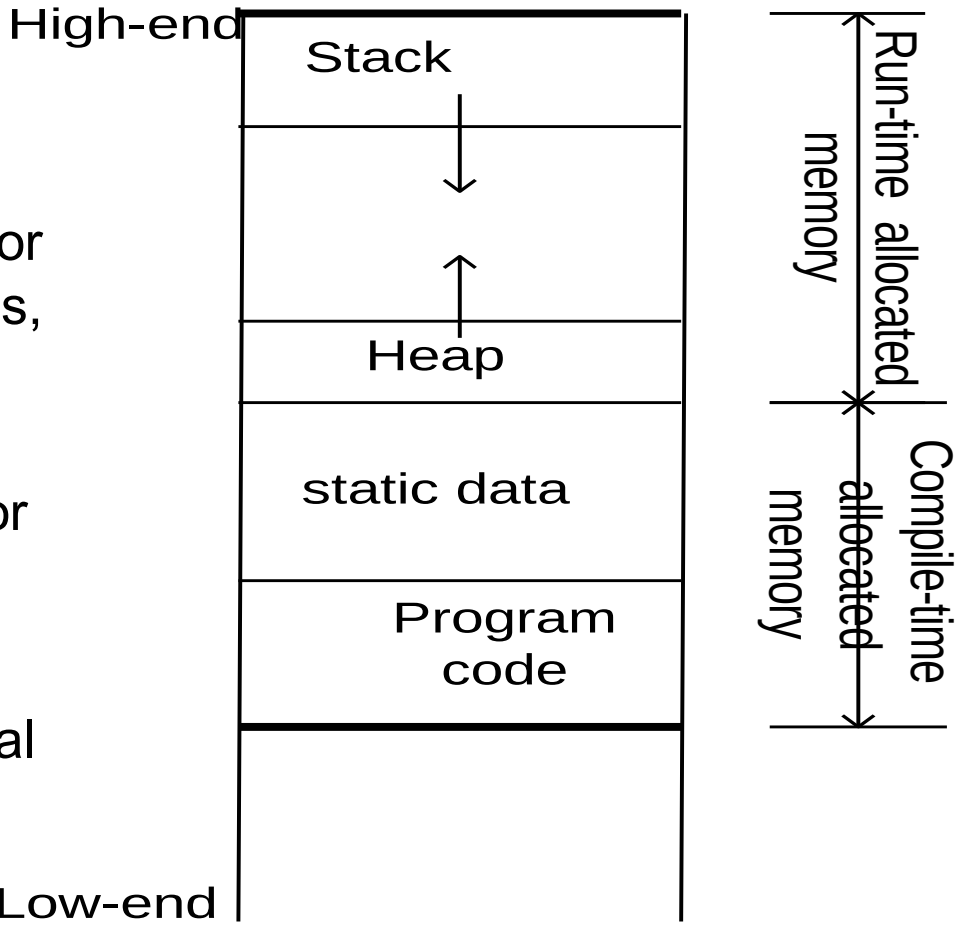
```
int  s;
```

The operating system allocates 4 bytes of memory to the program before its execution.

- ***Dynamic Allocation*:** Amount of memory space required is determined at run time (that is, after the program starts executing). Memory space is allocated to the program at run time too.

Allocation of Memory

- Memory map



STACK - This area is used for function calls return address, argument and local variables

HEAP – This area is used for Dynamic Memory Allocation

Static memory – reserved for global and static variables live

Dynamic Memory Allocation

- Dynamic allocation is useful when
  - arrays need to be created whose extent is not known until run time
  - complex structures of unknown size and/or shape need to be constructed as the program runs
  - objects need to be created and the constructor arguments are not known until run time
- Pointers are used for dynamic allocation of memory
- Use the operator **`new`** to dynamically allocate space
- Use the operator **`delete`** to free this space later

The **new** Operator

- If memory is available, the **new** operator allocates memory space for the requested object/array, and returns a pointer to (address of) the memory allocated.

```
Pointer = new   DataType;
```

```
Pointer = new   DataType  [IntExpression];
```

- If sufficient memory is not available, the **new** operator returns **NULL**.

The `delete` Operator

- The **`delete`** operator de-allocates the object or array currently pointed to by the pointer which was previously allocated at run-time by the **`new`** operator.
  - the freed memory space is returned to Heap
  - the pointer is then *considered* unassigned

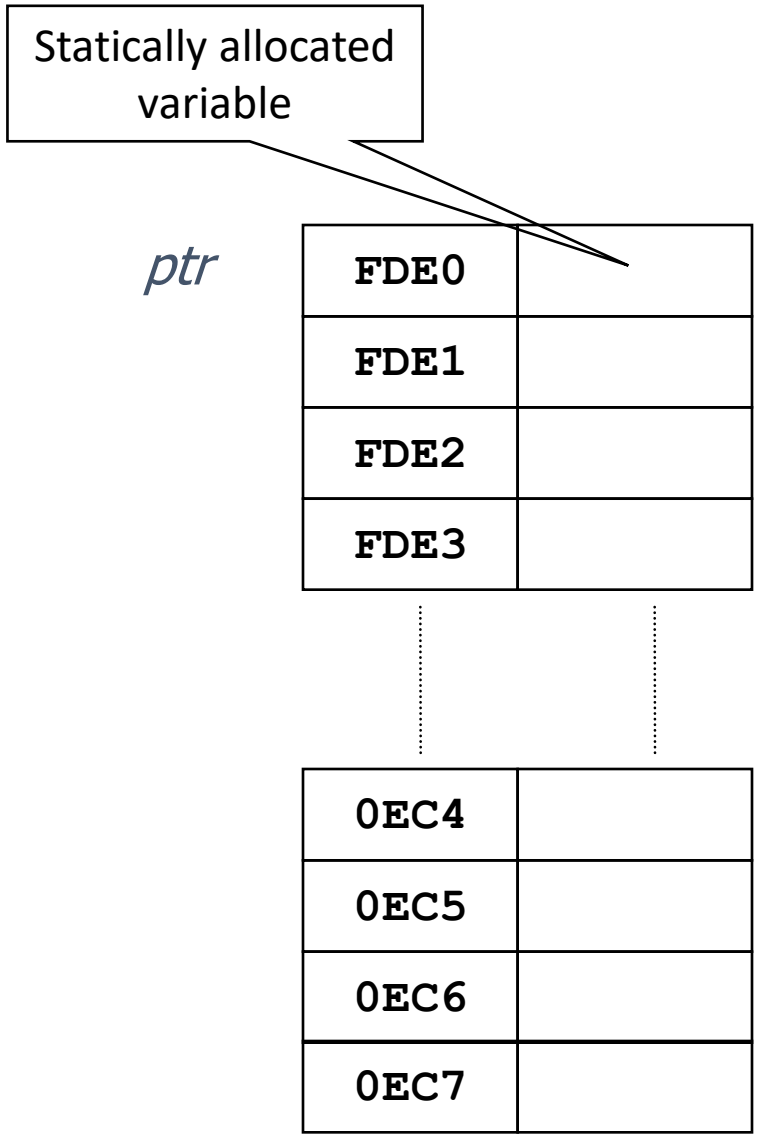
```
delete    Pointer;
```

```
delete   []   Pointer;
```

- If the value of the pointer is **`NULL`** there is no effect.

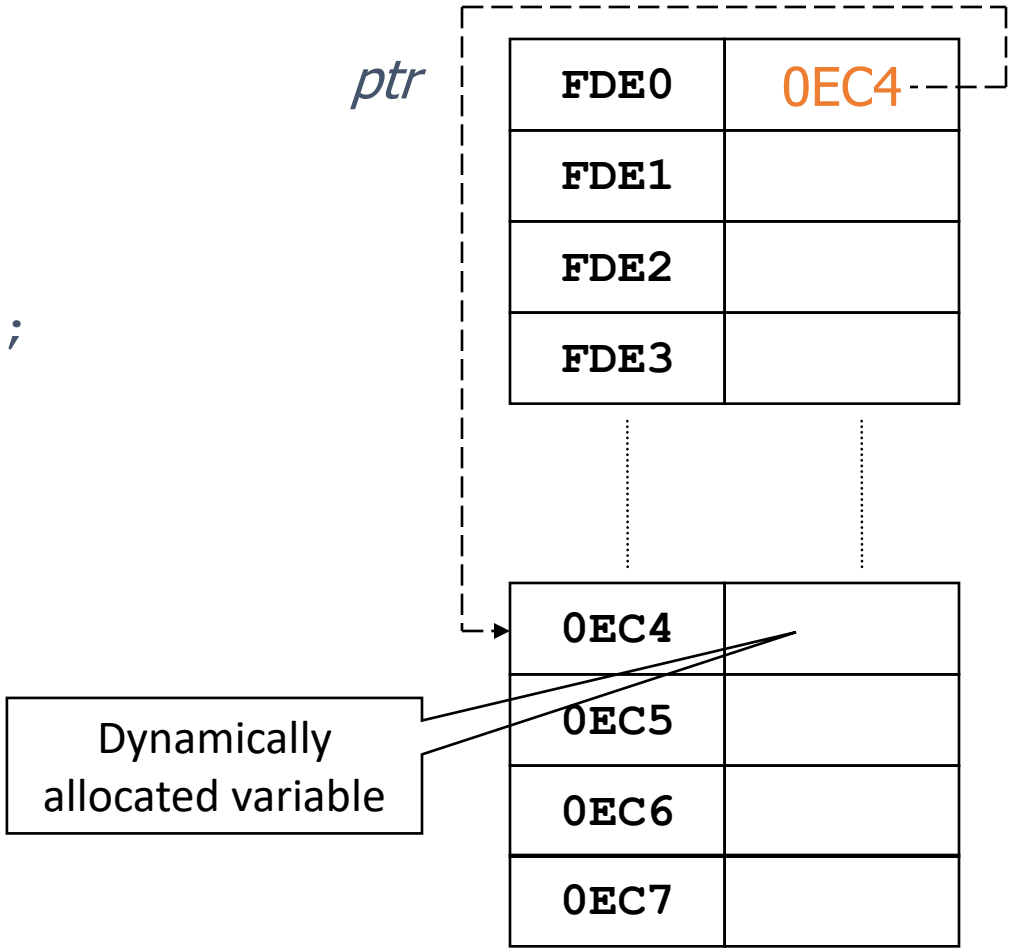
Example

```
int *ptr;
ptr = new int;
*ptr = 22;
cout << *ptr << endl;
delete ptr;
ptr = NULL;
```



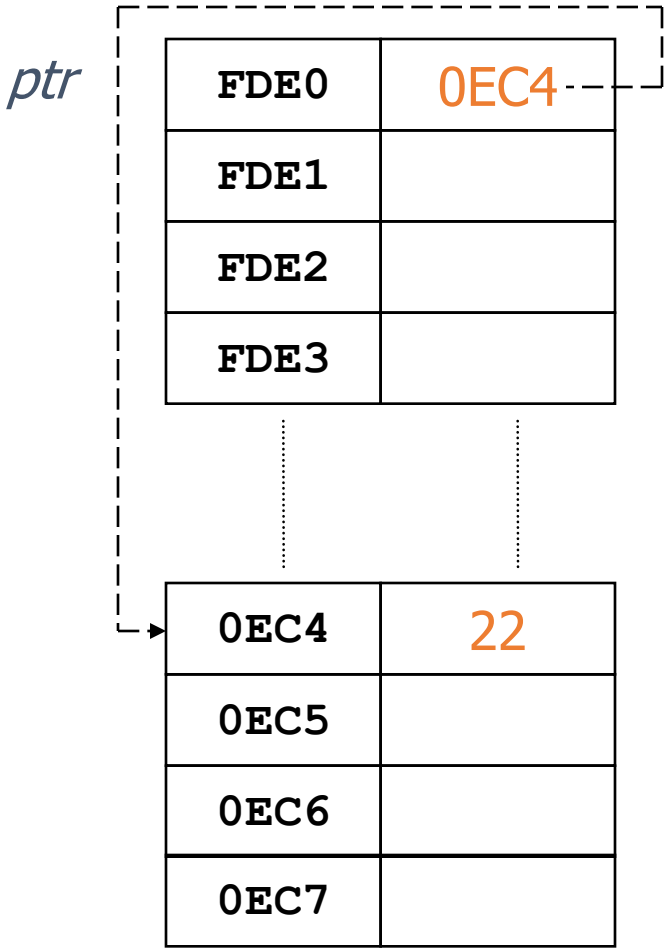
Example

```
int *ptr;
ptr = new int;
*ptr = 22;
cout << *ptr << endl;
delete ptr;
ptr = NULL;
```



Example

```
int *ptr;
ptr = new int;
*ptr = 22;
cout << *ptr << endl;
delete ptr;
ptr = NULL;
```

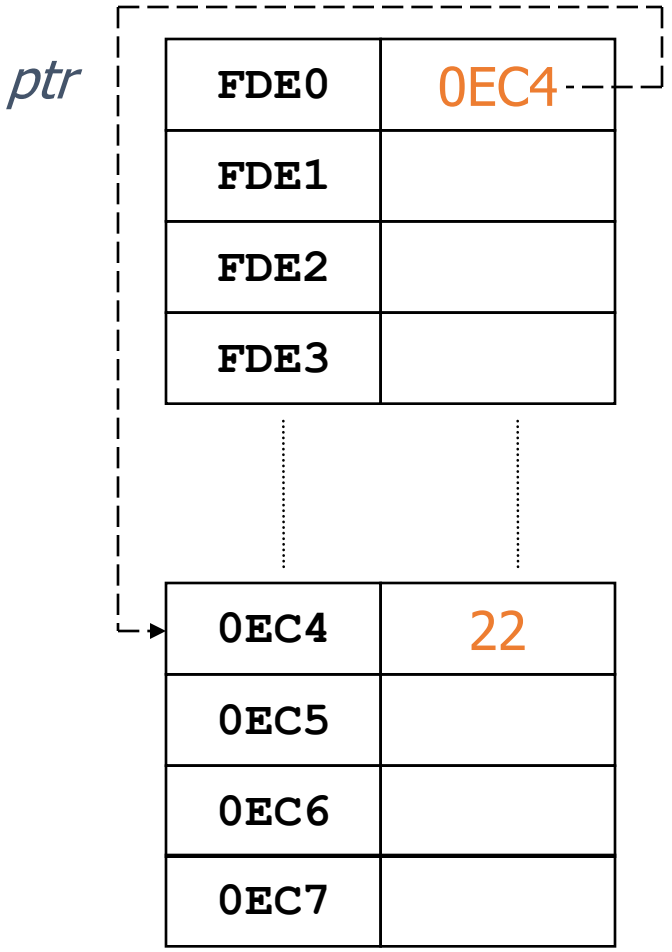


Example

```
int *ptr;
ptr = new int;
*ptr = 22;
cout << *ptr << endl;
delete ptr;
ptr = NULL;
```

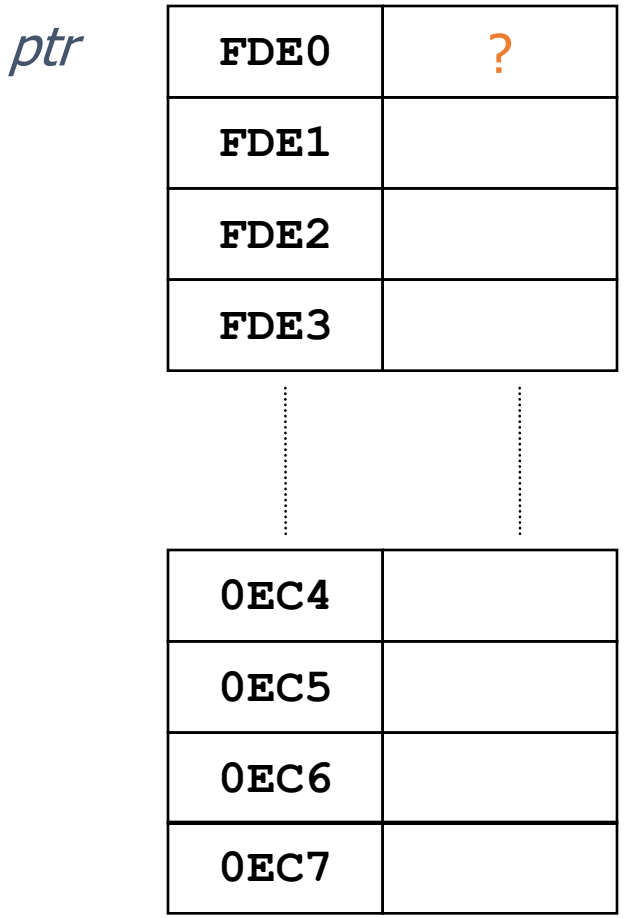
Output:

```
22
```



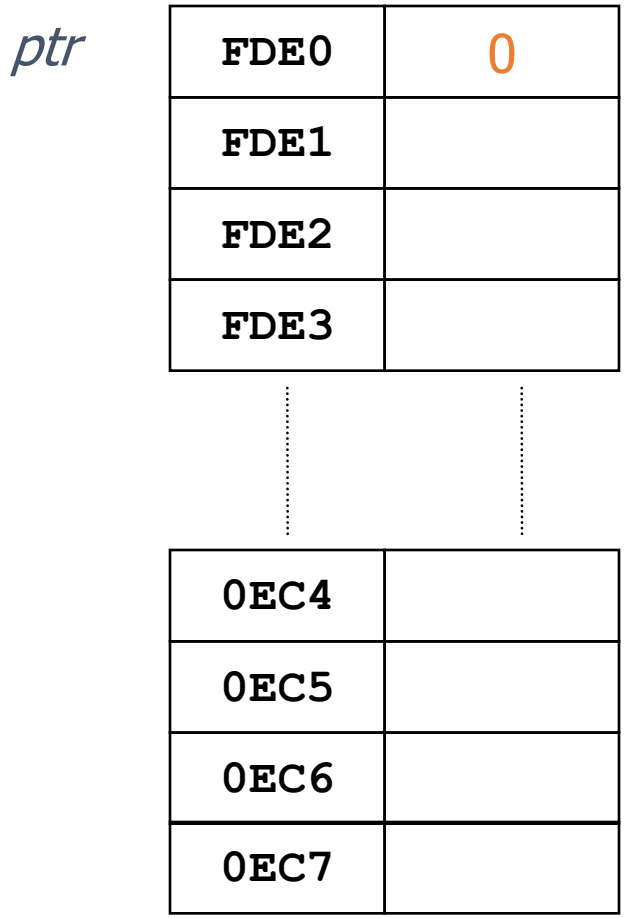
Example

```
int *ptr;
ptr = new int;
*ptr = 22;
cout << *ptr << endl;
delete ptr;
ptr = NULL;
```



Example

```
int *ptr;
ptr = new int;
*ptr = 22;
cout << *ptr << endl;
delete ptr;
ptr = NULL;
```



Example

```
int *grades = NULL;
int numberOfGrades;

cout << "Enter the number of grades: ";
cin >> numberOfGrades;
grades = new int[numberOfGrades];

for (int i = 0; i < numberOfGrades; i++)
   cin >> grades[i];

for (int j = 0; j < numberOfGrades; j++)
      cout << grades[j] << " ";

delete [] grades;
grades = NULL;
```

Dynamic Allocation of 2D Arrays

- A two dimensional array is really an array of arrays (rows).
- To dynamically declare a two dimensional array of `int` type, you need to declare a pointer to a pointer as:

```
int **matrix;
```

Dynamic Allocation of 2D Arrays

- To allocate space for the 2D array with **`r`** rows and **`c`** columns:
  - You first allocate the array of pointers which will point to the arrays (rows)

```
matrix = new int*[r];
```

  - This creates space for **`r`** addresses; each being a pointer to an **`int`**.
- Then you need to allocate the space for the 1D arrays themselves, each with a size of **`c`**

```
for(i=0; i<r; i++)
   matrix[i] = new int[c];
```

Example

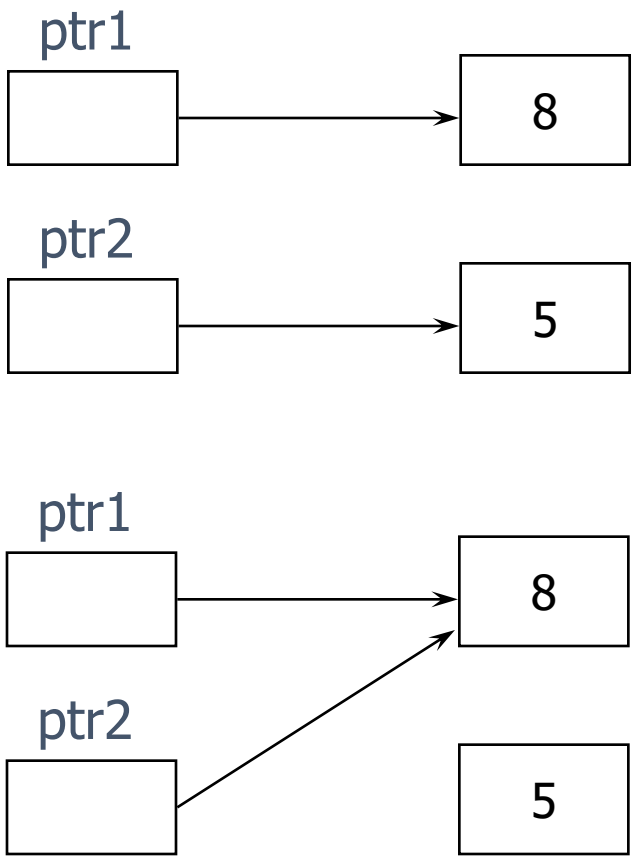
```
// create a 2D array dynamically
int rows, columns, i, j;
int **matrix;
cin >> rows >> columns;
matrix = new int*[rows];
for(i=0; i<rows; i++)
   matrix[i] = new int[columns];

 // deallocate the array
 for(i=0; i<rows; i++)
    delete [] matrix[i];
 delete [] matrix;
```

Memory Leak

- When you dynamically create objects, you can access them through the pointer which is assigned by the **`new`** operator
- Reassigning a pointer without deleting the memory it pointed to previously is called a memory leak
- It results in loss of available memory space

Memory Leak

```
int *ptr1 = new int;
int *ptr2 = new int;
*ptr1 = 8;
*ptr2 = 5;
ptr2 = ptr1;
```



Memory Leak

- Inaccessible memory location
  - Memory location that was allocated using **`new`**
  - There is no pointer that points to this memory space
- It is a logical error and causes memory leaks

Dangling Pointer

- A pointer that points to a memory location that has been de-allocated.
- The result of dereferencing a dangling pointer is unpredictable.

Dangling Pointer

```
int *ptr1 = new int;
int *ptr2;
*ptr1 = 8;
ptr2 = ptr1;
delete ptr1;
```